



REAL-TIME MULTIPLAYER 29 CARD GAME

Backend Architecture, Game Engine & System Design (Authoritative Engineering Document – v1)

Written and Documented by Sadik Al Sami

0. HOW TO READ THIS DOCUMENT

This document is written so that:

- A **first-time developer** can understand *what* is happening and *why*
- An **experienced engineer** can see *architectural intent and trade-offs*
- A **future contributor** can safely extend the system without breaking rules

This document defines:

- The **rules of the game**
- The **rules of the engine**
- The **rules of the system**

If something is not written here, it **must not exist in code**.

1. PROJECT GOALS (FROM 0 → 100)

Primary Goal

Build a **server-authoritative, real-time multiplayer 29 Card Game backend** that:

- Works with **guest users**
- Supports **room creation via link or code**

- Enforces **all standard + advanced 29 rules**
- Prevents cheating
- Handles reconnection safely
- Can be deployed to production
- Can scale in the future

Non-Goals (Important)

- No frontend UI logic here
 - No database initially
 - No AI / bots
 - No premature microservices
-

2. GAME OVERVIEW (FOR ANY DEVELOPER)

2.1 Players & Teams

Property	Value
Players	4
Teams	2
Team Formation	Opposite players
Seating	Circular
Play Direction	Counter-clockwise (default)

Seat Index Convention:

0 → 1 → 2 → 3 → back to 0

Team A: 0 & 2

Team B: 1 & 3

2.2 Deck & Card Model

Card Set

- Total cards: **32**
- Suits: ♣ ♦ ♥ ♠
- Ranks: J, 9, A, 10, K, Q, 8, 7

Rank Priority (High → Low)

$$J > 9 > A > 10 > K > Q > 8 > 7$$

Point Distribution

Card	Points
J	3
9	2
A	1
10	1
Others	0

Total round points = **28**

- **1 final round bonus = 29**

3. COMMUNICATION MODELS (CRITICAL CONCEPT)

This section corrects and clarifies **synchronous vs asynchronous communication**, which many engineers misunderstand.

3.1 Synchronous Communication (Strong Coupling)

Definition

Sender **waits for response**.

Failure propagates immediately.

Examples

- HTTP (REST)
- HTTP (GraphQL)
- WebSocket request–response (logical sync)

Characteristics

- Tight coupling
 - Request/response semantics
 - Immediate feedback
 - Easier mental model
-

3.2 Asynchronous Communication (Weak Coupling)

Definition

Sender **does not wait** for response.

Examples

- Message Queues (RabbitMQ, SQS)
- Pub/Sub (Kafka, Redis Streams)
- Server-Sent Events
- Event broadcasting via WebSockets

Characteristics

- Loose coupling
- Event-driven
- Better scalability

- More complex consistency handling

3.3 Where WebSocket's Fit (Important)

WebSocket's are **transport-level asynchronous**, but **application-level behavior can be either**:

Usage	Classification
Client sends intent → server responds immediately	Logically synchronous
Server broadcasts game state updates	Asynchronous
Spectator updates	Asynchronous

➡ **Our system uses a hybrid model:**

- **Synchronous intent validation**
- **Asynchronous state broadcasting**

3.4 Why WebSocket's for This Project

Requirement	HTTP	WebSocket
Real-time turn sync	✗	✓
Bi-directional	✗	✓
Low latency	✗	✓
Stateful connection	✗	✓

Learning Resources (Free)

- MDN WebSocket's Guide
- WebSocket vs HTTP articles
- Socket.IO official docs
- Real-time system design blogs

4. ARCHITECTURAL OVERVIEW

Client



WebSocket Gateway (transport only)



Room Manager (sessions)



Game Engine (rules + state)



In-Memory Store

4.1 Core Architectural Rule

| Only the Game Engine is allowed to mutate game state.

No exceptions.

5. GAME ENGINE – FROM FIRST PRINCIPLES

5.1 What the Engine IS

- A **pure rules engine**
 - Deterministic
 - Testable
 - Stateless outside its internal state
 - No networking
 - No storage
 - No timers
-

5.2 What the Engine is NOT

Capability	Allowed
WebSocket logic	✗
Database access	✗
UI logic	✗
Timers	✗
Player identity management	✗

5.3 Engine Responsibilities

- Deck creation & shuffle
- Card dealing
- Bidding enforcement
- Trump handling
- Single / Double / Redouble
- Pair / Marriage logic
- Under-trump enforcement
- Trick resolution
- Score calculation
- Cancellation rules
- Phase transitions

6. EXACT TYPESCRIPT INTERFACES (AUTHORITATIVE)

6.1 Card & Deck

```
export type Suit = 'CLUBS' | 'DIAMONDS' | 'HEARTS' | 'SPADES';

export type Rank = 'J' | '9' | 'A' | '10' | 'K' | 'Q' | '8' | '7';

export interface Card {
  suit: Suit;
  rank: Rank;
}
```

6.2 Player & Teams

```
export interface Player {
  id: string;
  seatIndex: 0 | 1 | 2 | 3;
  team: 'A' | 'B';
  hand: Card[];
  connected: boolean;
}
```

6.3 Game Phases

```
export type GamePhase =
  | 'WAITING'
  | 'DEALING'
  | 'BIDDING'
  | 'TRUMP_SELECTION'
  | 'MULTIPLIER'
  | 'PLAYING'
  | 'ROUND_END'
  | 'GAME_END'
  | 'CANCELLED';
```


6.4 Trump & Multipliers

```
export type TrumpType =  
  | { type: 'SUIT'; suit: Suit }  
  | { type: 'SEVENTH_CARD' }  
  | { type: 'JOKER' };  
  
export type Multiplier = 1 | 2 | 4;
```

6.5 Game State (ENGINE OWNED)

```
export interface GameState {  
  phase: GamePhase;  
  dealerSeat: number;  
  currentTurnSeat: number;  
  
  players: Player[];  
  
  deck: Card[];  
  bids: Map<string, number>;  
  winningBid?: number;  
  bidWinnerSeat?: number;  
  
  trump?: TrumpType;  
  trumpRevealed: boolean;  
  
  multiplier: Multiplier;  
  
  currentTrick: {  
    leadSuit?: Suit;  
    plays: { seat: number; card: Card }[];  
  };  
  
  tricksWon: {
```

```
A: Card[];  
B: Card[];  
};  
  
roundNumber: number;  
  
gamePoints: {  
  A: number;  
  B: number;  
};  
  
setScore: number;  
}
```

7. ROOM & SESSION MANAGEMENT

```
export interface Room {  
  id: string;  
  code: string;  
  players: Player[];  
  engine: GameEngine;  
  createdAt: number;  
  lastActivityAt: number;  
}
```

Rooms are:

- In memory
- Short lived
- Destroyed when idle

8. DETAILED MILESTONE PLAN

✓ MILESTONE 1 — FOUNDATIONS

Goal: A running backend with WebSocket connectivity.

Tasks

- ☐ Initialize Node + TypeScript
- ☐ Project folder structure
- ☐ ESLint + Prettier
- ☐ WebSocket server bootstrap
- ☐ Connection / disconnection handling
- ☐ Guest ID generation

Outcome

- ✓ Server accepts connections
 - ✓ No game logic yet
-

✓ MILESTONE 2 — ROOM SYSTEM

Goal: Players can create and join rooms.

Tasks

- ☐ Room registry (Map)
- ☐ Create room
- ☐ Join room via code
- ☐ Seat assignment
- ☐ Team assignment
- ☐ Room full validation
- ☐ Broadcast room state

Outcome

- ✓ 4 players per room

✓ Stable sessions

✓ MILESTONE 3 — GAME RULE SPECIFICATION

Goal: Finalize all 29 rules in code-ready form.

Tasks

- ☐ Encode card priority
- ☐ Encode point system
- ☐ Encode cancellation rules
- ☐ Encode advanced rules
- ☐ Rule validation checklist

Outcome

- ✓ Rulebook frozen
 - ✓ Engine can be written safely
-

✓ MILESTONE 4 — ENGINE CORE (OFFLINE)

Goal: Game engine works without WebSockets.

Tasks

- ☐ Deck creation
- ☐ Shuffle
- ☐ Dealing logic
- ☐ Phase machine
- ☐ Unit tests

Outcome

- ✓ Deterministic engine
 - ✓ Fully testable
-

✓ MILESTONE 5 — BIDDING & TRUMP

Goal: Complete pre-play logic.

Tasks

- ☐ Bidding order
- ☐ Bid validation
- ☐ Trump selection
- ☐ 7th card handling
- ☐ Joker trump

Outcome

- ✓ Legal bids only
 - ✓ Trump logic correct
-

✓ MILESTONE 6 — GAMEPLAY ENGINE

Goal: Full trick-based play.

Tasks

- ☐ Turn enforcement
- ☐ Lead suit logic
- ☐ Under-trump rules
- ☐ Trick winner resolution
- ☐ Score tracking

Outcome

- ✓ Full 8-round gameplay
-

✓ MILESTONE 7 — ADVANCED RULES

Goal: Full 29 compliance.

Tasks

- ☐ Double
- ☐ Redouble
- ☐ Single Hand
- ☐ Pair / Marriage
- ☐ Extra win/loss points

Outcome

- ✓ Tournament-ready rules
-

✓ MILESTONE 8 — WS INTEGRATION

Goal: Real-time multiplayer.

Tasks

- ☐ Intent → engine mapping
- ☐ State snapshot broadcasting
- ☐ Error propagation
- ☐ Hidden card enforcement

Outcome

- ✓ Playable online game
-

✓ MILESTONE 9 — STABILITY

Goal: Real-world reliability.

Tasks

- ☐ Reconnection handling
- ☐ Heartbeats
- ☐ Idle room cleanup

☐ Graceful shutdown

Outcome

✓ No broken games on refresh

✓ MILESTONE 10 — PRODUCTION & FUTURE

Goal: Deployment-ready and scalable.

Tasks

- ☐ Dockerfile
- ☐ Environment config
- ☐ Logging
- ☐ Redis design doc
- ☐ Scaling strategy

Outcome

✓ Production-capable backend

9. FUTURE UPGRADE PATHS

Feature	When
User accounts	After MVP
Match history	With DB
Spectators	Easy
AI players	Later
Mobile support	Frontend

10. FINAL NOTE

This document is:

- A **game rule constitution**
- A **backend architecture guide**
- A **training manual**
- A **scaling blueprint**

If followed strictly, the system will:

- Be correct
- Be extensible
- Be defendable in interviews
- Be production-ready